场景

之前外外出去面试，有被问到过一个这样的问题

组合索引a,b,c c=1 and a = 1 and b > 1，是否用到索引?

今天我们就以这个面试题作为出发点，来看怎么去查看有没有用到索引，用到了哪些索引，什么情况下索引会失效？为什么！以及怎么去优化？

怎么去判断 explain各个字段的含义

平时我们的日常工作中，想要分析你的sql语句或是表结构的性能怎样？我们一般会经常使用到哪个命令？explain

看到有同学提到了explain，那我们就用explain来分析下我们的面试题，验证下，到底有没有用到索引。

这里有张test表是我提前创建好的，里面就几个字段，主键id，以及a，b，c，d四个字段，在a,b,c三个字段上创建了一个组合索引index_unit，这张表量也不大，就七十多条，但是足够我们测试了，那我们用explain来验证下。

举例分析：

```
EXPLAIN SELECT SQL_NO_CACHE * FROM test WHERE  c=1 AND a =
1 AND b > 1;
```

信息 结果1 概况 状态

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	test	(Null)	range	index_unit	index_unit	10	(Null)	1	10	Using where; Using index

经过验证，很明显这条sql是用到了索引的，那我们仔细看下这个explain执行计划里面的内容，这么多字段都代表什么意思呢？我们平时虽然都有用到过，但是每个字段的含义我们都理解吗？

解释下explain中的各个字段的含义

那接下来，我们一个个的来看，各个字段都代表什么含义

select_type

select_type:根据这个字段可以看到当前查询类型，

select_type常用值：simple,primary,subquery,~~derived~~,union,union result

```
EXPLAIN SELECT * FROM test WHERE id = 1;--simple
SIMPLE      简单的select查询，查询中不包含子查询或者UNION

EXPLAIN SELECT * FROM test WHERE id = (SELECT MAX(id) FROM
test); --primary
PRIMARY     查询中若存在子查询的外层的表操作为PRIMARY
SUBQUERY    在SELECT或WHERE列表中包含了子查询

EXPLAIN SELECT id FROM test WHERE id = 1 UNION SELECT
test_id FROM test_1 WHERE id = 1;
DERIVED     在FROM列表中包含的子查询被标记为DERIVED
UNION       UNION操作中，查询中处于内层的SELECT
UNION RESULT    UNION操作的结果，id值通常为NULL
```

例子：

```
EXPLAIN SELECT SQL_NO_CACHE * from gp_student LEFT JOIN
gp_teacher on gp_student.teacher_no=gp_teacher.teacher_no

where teacher_age=18  limit 1000 ;


EXPLAIN SELECT SQL_NO_CACHE * FROM gp_student WHERE
teacher_no in
(
SELECT teacher_no FROM(
SELECT teacher_no FROM gp_teacher where teacher_age=18
 limit 1000
) a

)
```

table

table:这条数据是关于哪张表的查询

partitions

分区

type

type:表示访问类型

type的类型有很多，我们工作中常接触到的类型有: const > ref > range > index > all

解释下分别代表什么意思：

~~system :表只有一行（等于系统表）这是const类型的特列，平时不会出现，这个也可以忽略不计 const: 表示通过索引一次就找到了，~~

const： 通常情况下，如果将一个主键放置到where后面作为条件查询，mysql优化器就能把这次查询优化转化为一个常量。至于如何转化以及何时转化，这个取决于优化器。

```
EXPLAIN SELECT * FROM gp_teacher where teacher_no=1
```

~~eq_ref : 唯一性索引扫描，对于每个索引键，表中只有一条记录与之匹配，常见于主键或唯一索引的扫描~~

ref： 查找条件列使用了索引但是使用的不是主键或unique 。它返回所有匹配某个单独值的行，然而它可能会找到多个符合条件的行，所以它应该属于查找和扫描的混合体。(工作中能达到这个就很牛逼了)

```
EXPLAIN SELECT * FROM gp_teacher where teacher_age=18
```

咕泡出品

range：range指的是有范围的索引扫描，使用一个索引来选择行，key列显示使用了哪个索引，
一般就在你的where语句中出现between,<,>,in等类似的查询，这种范围扫描索引扫描比全表扫描要好，因为它只需要开始于索引的某一点，而结束语另一点，不用扫描全部索引。

```
EXPLAIN SELECT * FROM gp_teacher where teacher_age>18
 limit 1000;  -- 可以用到索引 range  因为只需要查1000数据，没有超过30%

EXPLAIN SELECT * FROM gp_teacher where teacher_age>18  -- 用不到索引，因为查询数据过多，优化器决定不用
```

index： 全索引扫描 ,index与ALL区别为index类型只遍历索引树，这通常比ALL快，
因为索引文件通常比数据文件小，（也就是说虽然all和Index都是读全表，但index是从索引中读取的，而all是从硬盘中读取）

```
EXPLAIN SELECT  teacher_name from gp_teacher where teacher_name='灰灰' limit 1000
```

all：全表扫描 ,将遍历全表以找到匹配的行。

```
EXPLAIN SELECT * FROM gp_teacher where teacher_address='灰灰'
```

互动提问，那么多类型中，哪个性能更好呢？const > ref > range > index > all

一般来说，你的SQL语句至少得保证达到range级别，最好达到ref。

possible_keys

possible_keys: 显示可能应用在这张表中的索引，一个或多个 查询涉及到的字段上若存在索引，则该索引将被列出，但不一定被查询实际使用。

```
EXPLAIN SELECT * FROM gp_teacher where teacher_age=18 and teacher_no>33  limit 1000
```

key

key :实际使用的索引，如果为NULL 则没有使用索引。查询中若使用了覆盖索引，则该索引仅出现在key列表中,优先使用普通索引，然后使用主键索引

key_len:表示索引中使用的字节数，可通过该列计算查询中使用索引的长度，在不损失精确性的情况下，长度越短越好。

ref

~~显示索引的哪一列被使用了，如果可能的话，是一个常量，哪些列被用于查找索引列上的值。~~

Filtered

百分比

rows

rows就是mysql认为必须要逐行去检查和判断的记录的条数 。

Extra

额外信息，它的显示十分重要。

1,Using index

表示相应的select操作中使用了覆盖索引，避免访问了表的数据行，效率不错!

如果同时出现using where,表明索引被用来执行索引键值查找, 代表使用了where过滤，并且数据不走索引树

如果没有同时出现using where ，表示索引用来读取数据而非执行查找动作。（teacher_age>18和teacher_age=18的区别）

咕泡出品

```
EXPLAIN SELECT teacher_no FROM gp_teacher where
teacher_age>18  limit 1000   -- Using where; Using index
EXPLAIN SELECT  teacher_no from gp_teacher where
teacher_name='灰灰' limit 1000   -- Using where; Using
index 代表数据从索引树拿
```

覆盖索引 在二级索引里面，如果select的数据列只用从索引中就能够取得，不必从数据区中读取，这时候使用的索引就叫做覆盖索引，这样就避免了回表。

2,Using where

使用了where过滤，要不要优化根据type一起考虑

```
EXPLAIN SELECT  * from gp_teacher where teacher_name='灰灰'
limit 1000    -- 使用了where过滤，并且数据不走索引树
EXPLAIN SELECT  teacher_no from gp_teacher where
teacher_address='灰灰' limit 1000
```

3,Using filesort

查询的时候用到了索引，但是排序的时候没有用到，导致sql用到文件排序

```
EXPLAIN SELECT  teacher_no from gp_teacher where
teacher_age>18 ORDER BY teacher_name DESC limit 1000  --
teacher_age用到了索引，但是teacher_name失效，因为联合索引前面一个
属于范围查询
```

4,Using index condition（索引条件下推）

使用二级索引,查找使用了索引，但是需要回表查询数据

```
EXPLAIN SELECT  * from gp_teacher where
teacher_address='灰灰' limit 1000   -- 这种是where,没有走到二
级索引

EXPLAIN SELECT * FROM gp_teacher where teacher_age>18
 limit 1000  -- Using index condition 走到二级索引，但是数据需
要回表查询
```

因为我们这里select * 查找的是所有字段,而我们的字段d是没有创建索引的,我们无法直接从索引树取数据,需要回表.

大家还记不记得索引下推：5.6以后完善的功能，只适用于二级索引。其实就是把过滤的动作在存储引擎做完，而不需要到Server层过滤。

总结下：explain执行计划中的各个字段都分析过了，type，key，extra尤为重要

怎么有没有用到索引？type key extra

现在大家是不是都知道怎么去分析一条sql有没有用到索引了，没问题的刷6

那我们再回到面试题！

EXPLAIN SELECT * FROM test WHERE c=1 AND a = 1 AND b > 1; 这条sql肯定用到了索引的！

但是在面试的过程中，我们如果只是简单说用到了是不够，我们需要知道原理，面试官可能不只是想要我们回答有没有用到，而是能具体到哪些字段用到

互动提问:到底哪些字段用到了？来，给出你的答案，大家刷起来

ok，很多同学都给出了自己的答案，那到底对不对呢？我暂时先不回答，我们先来了解下联合索引的原理

联合索引

我再问大家一个问题，我们建立联合索引的目的？有人知道吗？减少索引数量，因为每多一个索引，都会增加写操作的开销和磁盘空间的开销。

我们创建一个(a,b,c)的联合索引，相当于创建了(a),(a,b),(a,b,c)三个索引，命中其中任何一个都会走到了我们的联合索引。"一个顶三个"我们何乐而不为呢?

联合索引：可以支持***a | a,b| a,b,c*** 3种组合进行查找，但不支持 b,c进行查找 .为什么呢?

给大家举个例子：联合索引的结构与电话簿类似，人名由姓和名构成，电话簿首先按姓氏进行排序，然后按名字对有相同姓氏的人进行排序。如果你知道姓，电话簿将非常有用；如果你知道姓和名，电话簿则更为有用，但如果你只知道名不知道姓，电话簿将没有用处。

哪些场景会导致索引失效

ok，明白了原理，我们再回到面试题,来分析下哪个字段用到了索引，说道联合索引，我们平时听得最多的就是索引的最左匹配原则

什么是最左匹配？查询语句的where条件中的查询字段，有没有包含联合索引中最左边的列

这个定义可能还是比较难理解

互动提问:那我问下大家abc这个联合索引最左边的列是什么？

接着来分析我们的sql语句,我们到底该怎么看呢?首先我们来看下

```
SELECT *  FROM test WHERE  c=1 AND a = 1 AND b > 1
```

互动提问:where条件 有没有a ？

有a,那肯定是能走a索引的，那么如果是没有a的情况，会不会走索引呢？找个同学来回答下，我们来验证下

```
EXPLAIN SELECT * FROM test WHERE  c=1
```

从这条sql中我们可以看出如果where条件中没有a是不会走索引的，因为不满足最左匹配

那我们再看下另外一种情况，我把这个sql语句稍微改下

如果没有b,只有a c ，那有没有走索引？哪些字段走索引了？

```
SELECT * FROM test WHERE a= 1 and c=1
```

联合索引有没有走？我们是不是先检查where条件中有没有a， 有a，那是不是走了a，ab，abc中的a，所以肯定走了联合索引

那我们再看有没有 b ，没有b，那b就不会走索引

那我们接着看有没有c，有c，那c有没有走索引？我们现在只需要确认c到底有没有走索引，我们可以用个示例验证下

```
SELECT * FROM test WHERE a= 1;

SELECT * FROM test WHERE a= 1 AND c= 1;

EXPLAIN SELECT * FROM test WHERE a= 1 and c=1;
```

经过我们的验证，c并没有走索引，c 为什么没走索引呢？因为我们的索引是按a，ab，abc这样的顺序执行的，b既然已经中断，没有走索引，那b后面的c肯定不会走索引。

那我们再回到刚刚的面试题，a，b都走了索引，那么按照我们的理解，索引顺序执行，c是不是也要走索引呢？那到底C有没有走！！！同样我们是不是也可以用explain来验证下

```
EXPLAIN SELECT * FROM test WHERE  c=1 AND a =2 AND b > 1;


EXPLAIN SELECT * FROM test WHERE a =2 AND b > 1;  --有没有C，rows都是一样的，所以确定C是没走索引的
```

那么为什么明明没有中断，c还是没走索引呢？

当查询条件中出现某个列是范围查询的，存储引擎不能使用复合索引中该列其后的所有列。b>1是个范围，c所以是没走

那我说的到底对不对呢？同样我们也来验证下。我们把这个面试题改造下

反例：EXPLAIN SELECT * from test WHERE c=1 and a =2 AND b > 1;-- 范围查询row是6条

```
EXPLAIN SELECT * FROM test WHERE  c=1 AND a =2 AND b = 3;-- 这个是C走了索引的，因为不是范围
```

除了上面说的举例的>号以外，比如!= <> IS NOT NULL < 等范围条件查找，也会导致下一个索引列失效！！！

除了使下一个索引列失效外，还有当前索引列也失效的情况:

2.查询条件中某列使用LIKE % (如：'%ABC'），存储引擎不能使用索引中该列及其后的所有列

3.查询条件中某列使用函数的，存储引擎不能使用索引中该列及其后的所有列。

4，查询条件中某列使用类型转换的（包括显示的和隐示的），存储引擎不能使用索引中该列及其后的所有列。如：字符串类型的列NAME=3,就是隐示的类型转换，将INT型转换为字符串类型。
如果写为NAME='3',就不是类型转换。

另外还有一个比较容易忽略点，全部索引失效的情况:

1.数据超过30% (这只是个大概的值，并不准确)

2.最左的索引字段满足上面2.3.4 比如a

Mysql优化

查看索引是否失效

1.范围查询会导致当前索引列后的所有索引列失效 因为联合索引第一个字段都是一个范围了，就没法根据他下一个字段去搜索

比如！ = < > BETWEEN AND 等等

2. %开头，会使当前列以及后面所有列失效 因为当前列我都没法确定范围
3. 类型发生转换的时候，索引会失效，所以关联查询的时候，看下2个表的字段类型是否一
4. 使用函数运算时，自己都不在是自己了

建立合适索引

1.离散度低的不要建索引

2.尽量建立联合索引，减少索引树；优先建立where与orderby联合索引

尽量用覆盖索引，减少回表

查询不要用*

拿不准的查询语句先去线上预执行

减少表关联

每次返回数据一定要控制

咕泡出品